

Software Design Document (SDD)

LCD Driver Module

Module	LCD Driver
Version	1.0.0
Team	Steven McClellan, Vania Leal, Leonardo García, Melanie Picen
Date	02/06/2026
Institution	CETYS Universidad

1. Introduction

1.1. Purpose

This document describes the design and architecture of the LCD driver module for the STM32F411RE. It covers register-level communication with the HD44780 LCD controller via the PCF8574 I2C I/O expander, the 4-bit bus initialization sequence, and the character and cursor management API.

1.2. Scope

The LCD driver configures an HD44780-based 20x4 LCD display connected through a PCF8574 I2C expander at address 0x27. It depends on `i2c_driver` for all bus communication. The driver operates in 4-bit mode and exposes six public functions: `lcd_init`, `lcd_clear`, `lcd_set_cursor`, `lcd_put_char`, `lcd_print`, and `lcd_backlight`. Higher-level modules to initialize GPIO ports, configure pin modes, and perform atomic read, write, and toggle operations on individual GPIO pins. The driver supports ports A, B, C, D, E, and H, and covers pins 0 through 15 on each port.

1.3. Definitions and Acronyms

Term	Definition
GPIO	General Purpose Input/Output — configurable digital pins on a microcontroller.
STM32F411RE	ARM Cortex-M4 based microcontroller family by STMicroelectronics.
RCC	Reset and Clock Control — peripheral responsible for enabling clocks to GPIO ports.
AHB1	Advanced High-performance Bus 1 — bus to which GPIO peripherals are connected on STM32F4.
MODER	GPIO Mode Register — configures each pin as input, output, alternate function, or analog.
ODR	Output Data Register — holds the output state of each GPIO pin.

IDR	Input Data Register — reflects the current logic level of each GPIO pin.
BSRR	Bit Set/Reset Register — allows atomic set and reset of individual GPIO output pins.
CMSIS	Cortex Microcontroller Software Interface Standard — vendor-independent hardware abstraction layer.
SRS	Software Requirements Specification — document defining the functional requirements for this module.
SDD	Software Design Document — this document.
API	Application Programming Interface — set of public functions exposed by the driver.
FR	Functional Requirement — a requirement that specifies a system behavior.
NFR	Non-Functional Requirement — a requirement that specifies a quality attribute of the system.

1.4. Requirements

All functional requirements specific to this module are defined and allocated in the Software Requirements Specification (SRS) for the GPIO driver module (Version 1.2, authored by Carlos Villarreal, CETYS Universidad). The SRS is the authoritative source for the behavioral requirements that this design document implements. The table below summarizes the traceability between each SRS functional requirement and the corresponding public API function implemented in this driver.

Req. ID	Function	Requirement Statement
FR-1	lcd_init()	The system shall configure all GPIO ports to a default state.
FR-2	lcd_clear()	The system shall enable clocking for the specified port.
FR-3	lcd_set_cursor()	The system shall allow configuration of pin direction and mode.
FR-4	lcd_put_char()	The system shall drive the specified pin to a high state.
FR-5	lcd_print()	The system shall drive the specified pin to a low state.
FR-6	lcd_backlight()	The system shall invert the current state of the specified pin.
FR-7	()	The system shall return the digital state of the specified pin.

2. System Architecture

2.1. High-Level Design

The LCD driver follows a layered architecture separating application logic from I2C bus and HD44780 protocol details. The four layers are described below:

Layer	Description
Application Layer	Higher-level software modules that call the GPIO driver public API functions to control hardware behavior without direct register access.
GPIO Driver Layer	Implements the public API declared in <code>GPIO_stm32f4.h</code> . Validates all input parameters, enables peripheral clocks via RCC, and performs direct read/write operations on GPIO hardware registers through the <code>GPIO_RegDef_t</code> and <code>RCC_RegDef_t</code> register map structures.
Hardware Layer	Actual STM32F4 microcontroller GPIO peripheral registers (MODER, ODR, IDR, BSRR, etc.) mapped at fixed memory addresses via the AHB1 peripheral bus.

2.2. Module Interfaces

The LCD driver exposes its functionality exclusively through the public header file `lcd_i2c.h`. The module has the following interface characteristics:

- **Public API:** All user-callable functions are declared in `lcd_i2c.h`: `lcd_init`, `lcd_clear`, `lcd_set_cursor`, `lcd_put_char`, `lcd_print`, `lcd_backlight`.
- **Internal implementation:** Four static helpers (`pcf8574_write`, `lcd_pulse_enable`, `lcd_send_nibble`, `lcd_send_byte`) are defined in `lcd_i2c.c` and not visible to external modules.
- **Dependencies:** `lcd_i2c.h` depends on `i2c_driver.h` for `i2c_writeDevice()` and on `<stdint.h>` for fixed-width types.
- **Hardware dependencies:** I2C1 peripheral (PB8=SCL, PB9=SDA), PCF8574 I2C expander, and HD44780 LCD controller.

3. Detailed Design

3.1. Class Diagram

The figure below shows the static structure of the LCD driver: `lcd_handle_t`, the six public functions, the four internal helpers, and their dependencies on `i2c_driver`, `PCF8574`, and `HD44780`. Public members are marked with a filled circle (●) and private/internal members with an open circle (○).

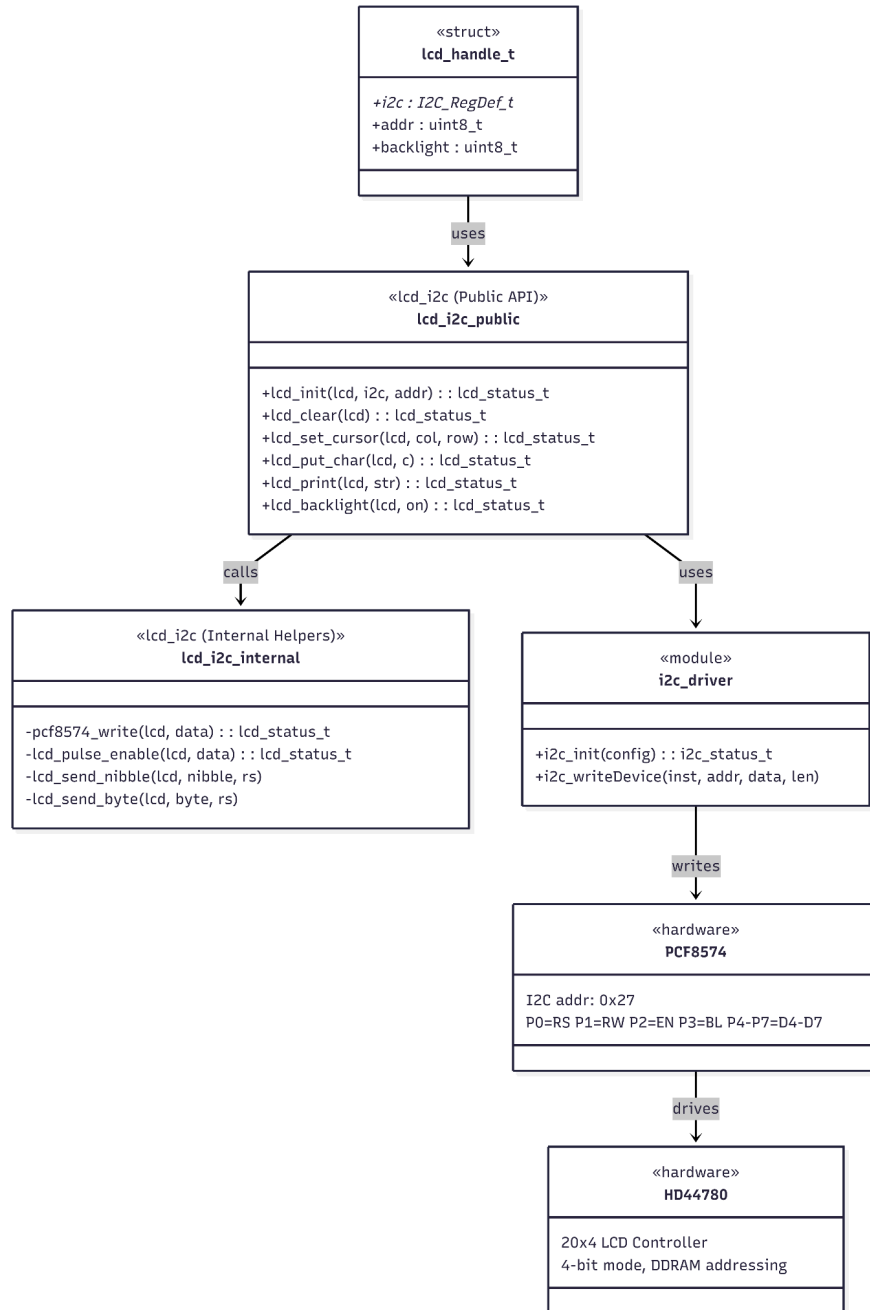


Figure 1. Class Diagram — LCD Driver

3.2. Module Behavior

The following diagrams describe the behavioral design of the key driver operations.agram (interaction between caller and driver) and an activity diagram (internal control flow including input validation).

lcd_init — HD44780 Initialization (FR-1)

lcd_init() performs the mandatory HD44780 power-on sequence: 50ms wait, three 8-bit mode nibbles (0x3), switch to 4-bit (0x2), then function, display, clear, and entry-mode commands.. Port A is excluded from the MODER reset to preserve the SWD debug interface.

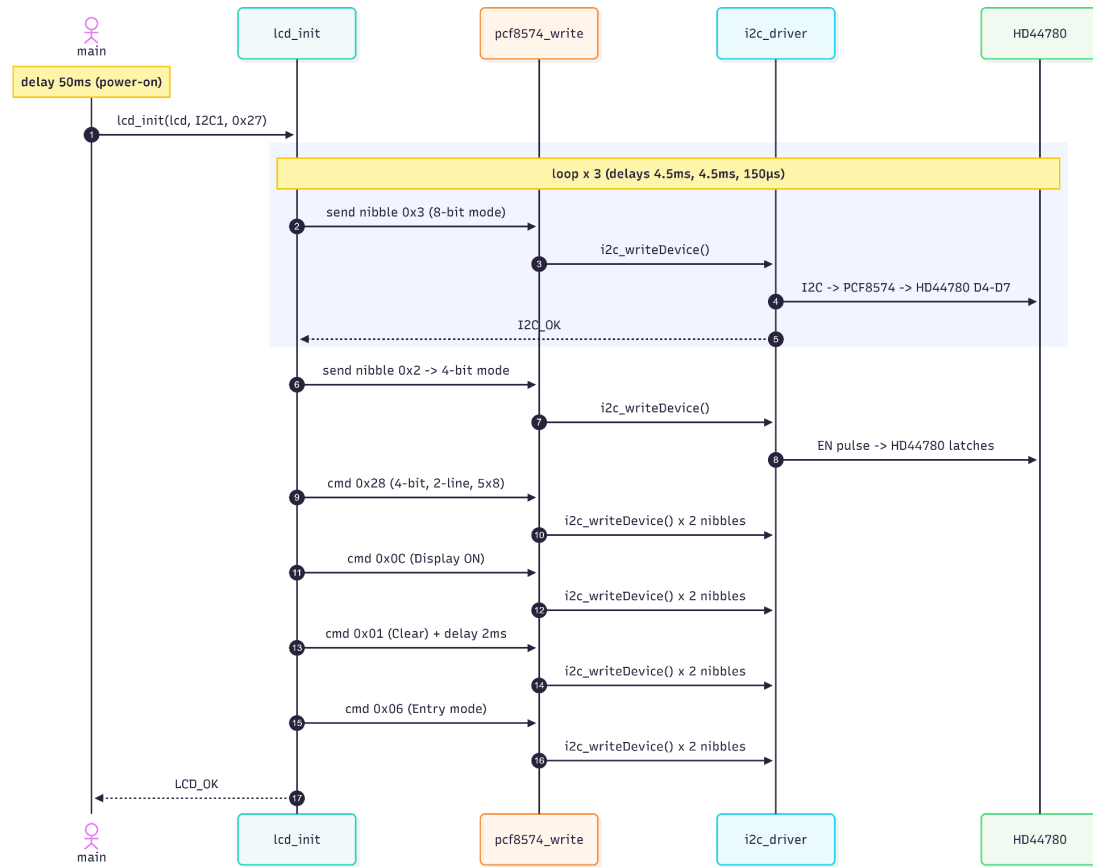


Figure 2. Sequence Diagram — lcd_init

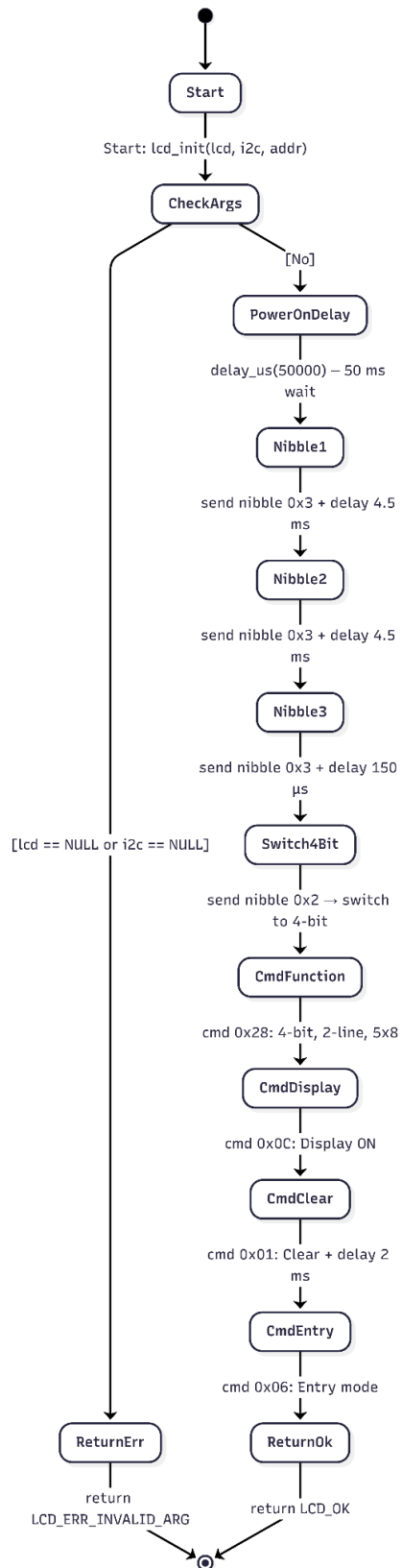


Figure 3. Activity Diagram — lcd_init

lcd_print / lcd_put_char — String and Character Output (FR-5)

lcd_print() iterates a null-terminated string calling lcd_send_data() per character. lcd_put_char() sends a single character with RS=1. Returns GPIO_ERR_PORT immediately if the port is out of range.

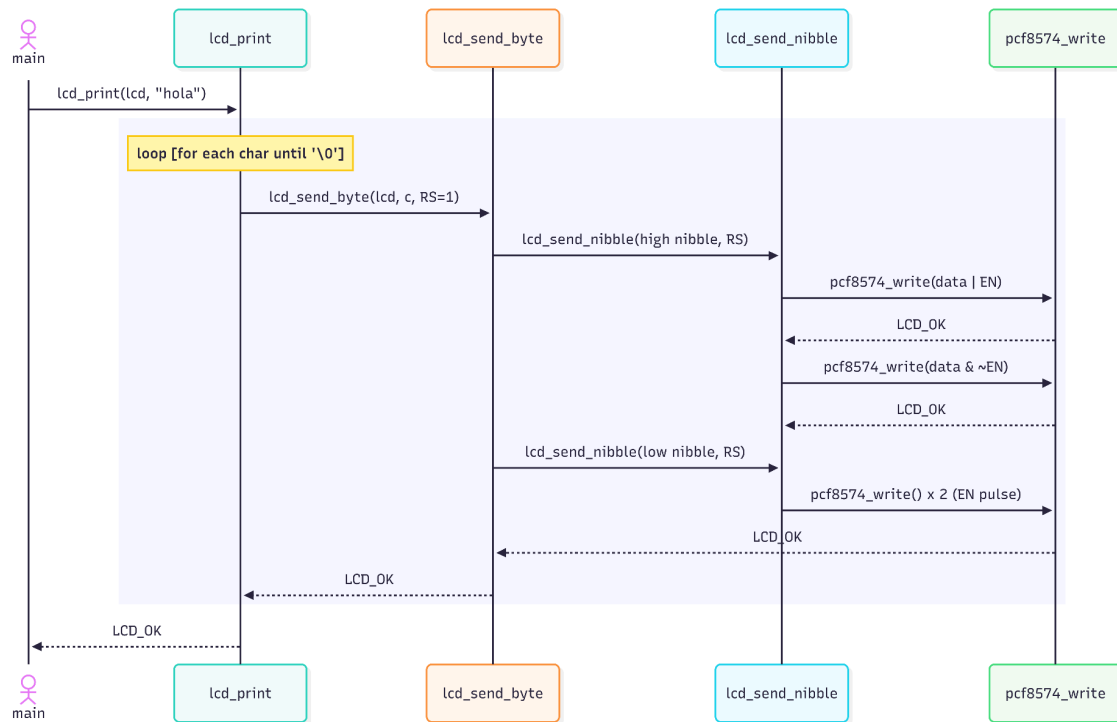


Figure 4. Sequence Diagram — lcd_print

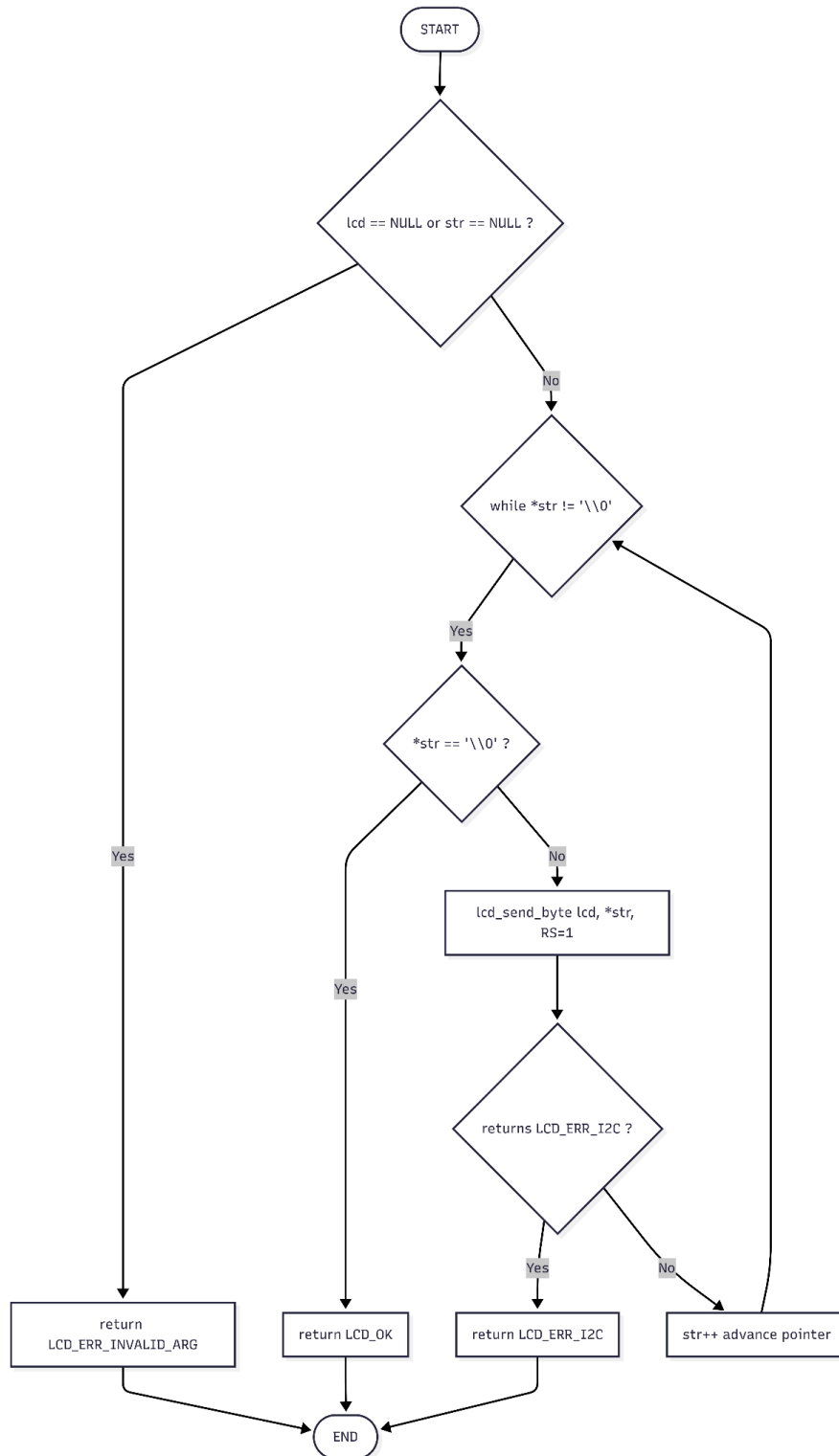


Figure 5. Activity Diagram — `lcd_print`

pcf8574_write / lcd_pulse_enable — I2C and EN Pulse Protocol (FR-2, FR-3)

pcf8574_write() sends one byte to PCF8574 via i2c_writeDevice(). lcd_pulse_enable() raises EN high 1us, lowers it so HD44780 latches, waits 50us.r. The 2-bit field for the target pin is cleared first, then the new mode value is OR'd in, leaving all other pin configurations intact.

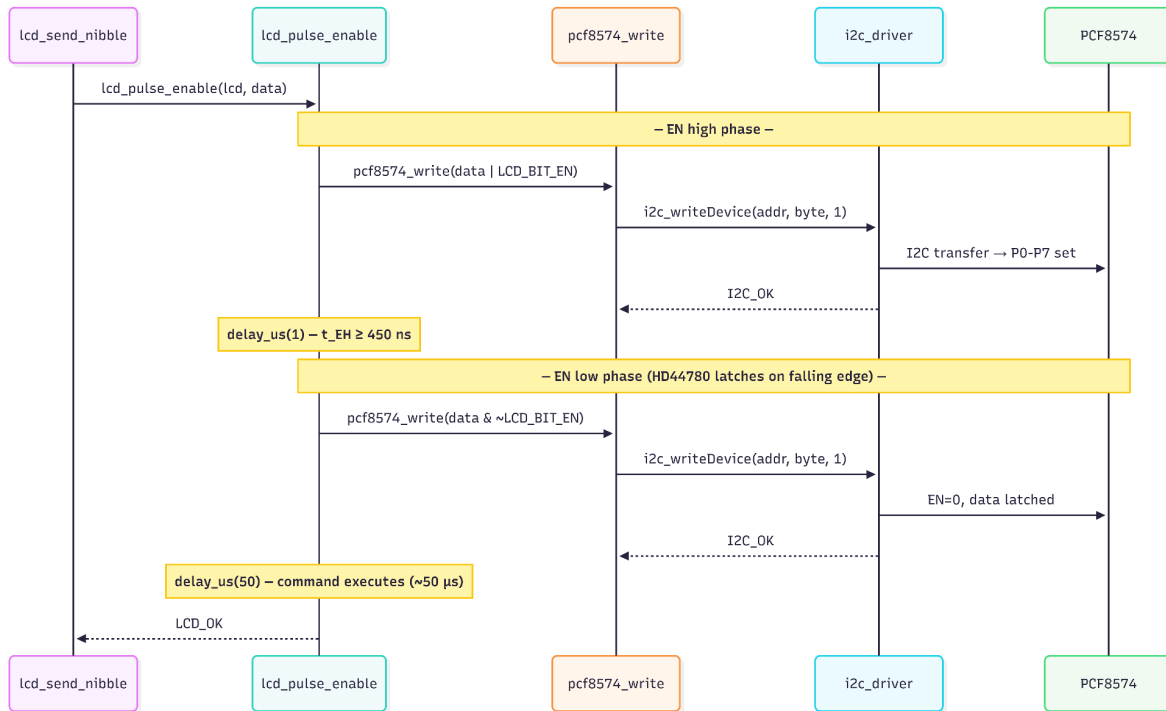


Figure 6. Sequence Diagram — `pcf8574_write` / `lcd_pulse_enable`

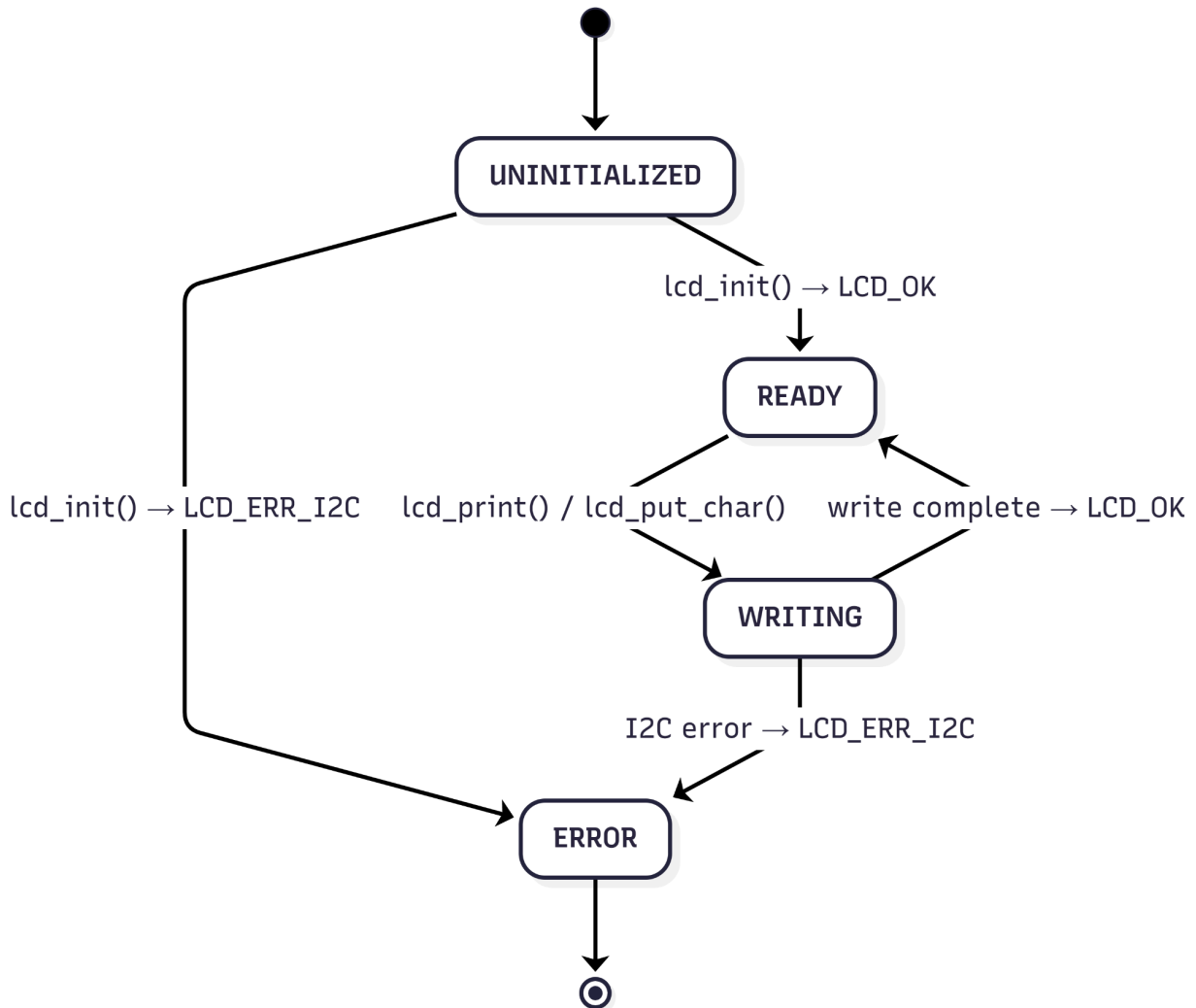


Figure 7. State Diagram — LCD Driver Lifecycle

3.3. LCD Driver

The LCD driver is implemented in `lcd_i2c.h` and `lcd_i2c.c`. It avoids dynamic memory allocation. All state is stored in the `lcd_handle_t` struct passed by the caller.

Internal Helpers

Four static helper functions are defined in `lcd_i2c.c` and not visible outside:

- `pcf8574_write()`: single point of I2C access via `i2c_writeDevice()`.
- `lcd_pulse_enable()`: EN high 1us, EN low 50us.
- `lcd_send_nibble()`: packs nibble in bits 7:4, ORs RS and BL, calls `lcd_pulse_enable()`.
- `lcd_send_byte()`: calls `lcd_send_nibble()` twice — high nibble then low nibble.

Initialization Sequence

`lcd_init()`: 50ms wait, 3x nibble 0x3, nibble 0x2, then cmd 0x28, 0x0C, 0x01, 0x06.

Cursor and Display Control

lcd_clear(): sends 0x01 command and waits 2ms. lcd_set_cursor() uses DDRAM bases 0x00, 0x40, 0x14, 0x54 for rows 0-3.

Output Functions

lcd_put_char(): sends one ASCII character with RS=1. lcd_print(): iterates string calling lcd_send_data() per character. Does not wrap rows automatically — caller must call lcd_set_cursor() before each row. The backlight state is stored in lcd_handle_t and ORed into every PCF8574 byte automatically.

- wrap rows automatically. Caller must call lcd_set_cursor() before each row.o_clk_masks[].

4. Application Programming Interfaces

4.1. Data Types and Macros

Data Structures

lcd_handle_t: struct with I2C_RegDef_t *i2c, uint8_t addr, uint8_t backlight.vent compiler optimization of hardware register accesses.

Member	Type	Notes / Definition
MODER	volatile uint32_t	Offset 0x00 — configures each pin: input(00), output(01), alt-fn(10), analog(11).
OTYPER	volatile uint32_t	Offset 0x04 — output type: push-pull(0) or open-drain(1).
OSPEEDR	volatile uint32_t	Offset 0x08 — controls slew rate / output speed per pin.
PUPDR	volatile uint32_t	Offset 0x0C — configures pull-up(01) or pull-down(10) resistors.
IDR	volatile uint32_t	Offset 0x10 — read-only; reflects the current logic level on each pin.
ODR	volatile uint32_t	Offset 0x14 — read/write; controls output state of each pin.
BSRR	volatile uint32_t	Offset 0x18 — write-only; bits[15:0] set pins, bits[31:16] reset pins atomically.
LCKR	volatile uint32_t	Offset 0x1C — locks pin configuration until next reset.
AFR[2]	volatile uint32_t	Offset 0x20 — alternate function: AFR[0] for pins 0–7, AFR[1] for pins 8–15.

lcd_status_t: LCD_OK (0), LCD_ERR_INVALID_ARG (-1), LCD_ERR_I2C (-2).sed by this driver.

Member	Type	Notes / Definition
CR	volatile uint32_t	Clock control register.
PLLCFGR	volatile uint32_t	PLL configuration register.

CFGR	volatile uint32_t	Clock configuration register.
CIR	volatile uint32_t	Clock interrupt register.
AHB1RSTR	volatile uint32_t	AHB1 peripheral reset register.
AHB2RSTR	volatile uint32_t	AHB2 peripheral reset register.
RESERVED0[2]	volatile uint32_t	Reserved — must not be accessed.
APB1RSTR	volatile uint32_t	APB1 peripheral reset register.
APB2RSTR	volatile uint32_t	APB2 peripheral reset register.
RESERVED1[2]	volatile uint32_t	Reserved — must not be accessed.
AHB1ENR	volatile uint32_t	AHB1 peripheral clock enable — bit 0 = GPIOA, bit 1 = GPIOB, ..., bit 7 = GPIOH.

Macros

LCD_PCF8574_ADDR = 0x27U, LCD_COLS = 20U, LCD_ROWS = 4U, LCD_BIT_RS/RW/EN/BL, LCD_CMD_*, LCD_ROW0-3_ADDR.

Enumerator	Value	Definition
GPIO_PORT_A	0	Port A
GPIO_PORT_B	1	Port B
GPIO_PORT_C	2	Port C
GPIO_PORT_D	3	Port D
GPIO_PORT_E	4	Port E
GPIO_PORT_H	5	Port H
GPIO_PORT_MAX	6	Sentinel — boundary validation; not a valid port.

register.

Enumerator	Value	Definition
GPIO_MODE_INPUT	0x00	Digital input — pin samples the external voltage level.
GPIO_MODE_OUTPUT	0x01	Digital output — pin is driven by ODR or BSRR.
GPIO_MODE_ALTFN	0x02	Alternate function — pin controlled by a peripheral (UART, SPI, I2C, etc.).
GPIO_MODE_ANALOG	0x03	Analog mode — connected to ADC/DAC; digital buffers disabled.
GPIO_MODE_MAX	4	Sentinel — boundary validation; not a valid mode.

Macro	Value	Definition
PERIPH_BASE	0x40000000UL	Base address of all STM32F4 peripherals.
AHB1PERIPH_BASE	PERIPH_BASE + 0x00020000UL	Base address of the AHB1 peripheral bus.
GPIOA_BASE	AHB1PERIPH_BASE + 0x0000UL	Base address of GPIO port A registers.
GPIOB_BASE	AHB1PERIPH_BASE + 0x0400UL	Base address of GPIO port B registers.
GPIOC_BASE	AHB1PERIPH_BASE + 0x0800UL	Base address of GPIO port C registers.
GIOD_BASE	AHB1PERIPH_BASE + 0x0C00UL	Base address of GPIO port D registers.
GPIOE_BASE	AHB1PERIPH_BASE + 0x1000UL	Base address of GPIO port E registers.
GPIOH_BASE	AHB1PERIPH_BASE + 0x1C00UL	Base address of GPIO port H registers.
RCC_BASE	AHB1PERIPH_BASE + 0x3800UL	Base address of the RCC peripheral registers.
GPIOA ... GPIOH	(GPIO_RegDef_t *) GPIOx_BASE	Typed pointer macros for direct register access per port.
RCC	(RCC_RegDef_t *) RCC_BASE	Typed pointer macro for RCC direct register access.
RCC_AHB1ENR_GPIOAEN	(1U << 0)	Bit mask to enable GPIOA clock in RCC->AHB1ENR.
RCC_AHB1ENR_GPIOBEN	(1U << 1)	Bit mask to enable GPIOB clock in RCC->AHB1ENR.
RCC_AHB1ENR_GPIOCEN	(1U << 2)	Bit mask to enable GPIOC clock in RCC->AHB1ENR.
RCC_AHB1ENR_GPIODEN	(1U << 3)	Bit mask to enable GPIOD clock in RCC->AHB1ENR.
RCC_AHB1ENR_GPIOEEN	(1U << 4)	Bit mask to enable GPIOE clock in RCC->AHB1ENR.
RCC_AHB1ENR_GPIOHEN	(1U << 7)	Bit mask to enable GPIOH clock in RCC->AHB1ENR.
GPIO_PIN_MAX	15U	Maximum valid pin number (inclusive).

4.2. Error Codes

All public LCD API functions return `lcd_status_t`: LCD_OK (0), LCD_ERR_INVALID_ARG (-1), LCD_ERR_I2C (-2).e listed below:

Code	Value	Definition
GPIO_OK	0	Operation completed successfully.
GPIO_ERR_PORT	-1	The port parameter is invalid ($\geq$ GPIO_PORT_MAX).
GPIO_ERR_PIN	-2	The pin parameter is invalid ($>$ GPIO_PIN_MAX = 15).
GPIO_ERR_MODE	-3	The mode parameter is invalid ($\geq$ GPIO_MODE_MAX).
GPIO_ERR_NULL	-4	A required pointer is NULL (applies to gpio_readPin state pointer).

4.3. Function Definitions

gpio_init

Function Name	gpio_init
Syntax	void gpio_init(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	void
Description	Enables peripheral clocks for all six supported GPIO ports (A, B, C, D, E, H) simultaneously by setting the corresponding bits in RCC->AHB1ENR. Additionally resets the MODER register to 0x00000000 for ports B, C, D, E, and H. Port A's MODER is intentionally not reset to preserve SWD debug interface on PA13 and PA14.
Constraints	No parameter validation performed. Port A MODER is not reset. Must be called before any other GPIO driver function if all ports are to be initialized at once.
Available via	GPIO_stm32f4.h

gpio_initPort

Function Name	gpio_initPort	
Syntax	GPIO_Status_t gpio_initPort(GPIO_Port_t port)	
Parameters (in)	port	GPIO_Port_t — selects the GPIO port whose clock will be enabled (GPIO_PORT_A through GPIO_PORT_H).

Parameters (inout)	None —	
Parameters (out)	None —	
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT if port >= GPIO_PORT_MAX.
Description	Enables the peripheral clock for a single GPIO port using the gpio_clk_masks[] lookup table to set the correct bit in RCC->AHB1ENR. Sends Clear Display command (0x01), waits 2ms, moves cursor to (0,0).	
Constraints	Does not reset or configure pin modes; only enables the clock. The caller must call gpio_setPinMode separately to configure individual pins.	
Available via	GPIO_stm32f4.h	

gpio_setPinMode

Function Name	gpio_setPinMode	
Syntax	GPIO_Status_t gpio_setPinMode(GPIO_Port_t port, uint8_t pin, GPIO_Mode_t mode)	
Parameters (in)	port	GPIO_Port_t — target GPIO port.
Parameters (in)	pin	uint8_t — target pin number (0–15).
Parameters (in)	mode	GPIO_Mode_t — desired pin mode (INPUT, OUTPUT, ALTFN, ANALOG).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT, GPIO_ERR_PIN, or GPIO_ERR_MODE if any parameter is invalid.
Description	Configures the mode of an individual GPIO pin by writing to the 2-bit field in MODER at bit position (pin × 2). Clears the field first using bitmask AND, then sets it using OR with the mode value. col (0-19) and row (0-3) validated. Sends Set DDRAM Address command.	
Constraints	Does not configure PUPDR, OTyPER, or AFR. The port clock must be enabled before calling this function.	

Available via	GPIO_stm32f4.h
----------------------	----------------

gpio_setPin

Function Name	gpio_setPin	
Syntax	GPIO_Status_t gpio_setPin(GPIO_Port_t port, uint8_t pin)	
Parameters (in)	port	GPIO_Port_t — target GPIO port.
Parameters (in)	pin	uint8_t — target pin number (0–15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT or GPIO_ERR_PIN if parameters are invalid.
Description	Atomically drives a GPIO output pin high by writing 1 to bit position pin in BSRR bits [15:0]. The BSRR register guarantees an atomic operation with no intermediate glitch state.	
Constraints	The pin must be configured as output before calling. Calling on an input-configured pin has no visible effect. Sends one ASCII character at current cursor position with RS=1.	
Available via	GPIO_stm32f4.h	

gpio_clearPin

Function Name	gpio_clearPin	
Syntax	GPIO_Status_t gpio_clearPin(GPIO_Port_t port, uint8_t pin)	
Parameters (in)	port	GPIO_Port_t — target GPIO port.
Parameters (in)	pin	uint8_t — target pin number (0–15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT or

		GPIO_ERR_PIN if parameters are invalid.
Description	Atomically drives a GPIO output pin low by writing 1 to bit position (pin + 16) in BSRR bits [31:16]. Iterates str calling lcd_send_data() per character. Stops on null terminator or I2C error.	
Constraints	Same constraints as gpio_setPin: pin must be in output mode and port clock must be enabled.	
Available via	GPIO_stm32f4.h	

gpio_togglePin

Function Name	gpio_togglePin	
Syntax	GPIO_Status_t gpio_togglePin(GPIO_Port_t port, uint8_t pin)	
Parameters (in)	port	GPIO_Port_t — target GPIO port.
Parameters (in)	pin	uint8_t — target pin number (0–15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT or GPIO_ERR_PIN if parameters are invalid.
Description	Inverts the current output state of a GPIO pin by XOR-ing bit pin in the ODR register (ODR ^= 1 << pin). Updates backlight field and immediately sends zero-data byte to PCF8574.	
Constraints	Non-atomic read-modify-write on ODR. If an interrupt or concurrent context modifies ODR for the same port, a race condition may occur.	
Available via	GPIO_stm32f4.h	

gpio_readPin

Function Name	gpio_readPin	
Syntax	GPIO_Status_t gpio_readPin(GPIO_Port_t port, uint8_t pin, uint8_t *state)	
Parameters (in)	port	GPIO_Port_t — target GPIO port.
Parameters (in)	pin	uint8_t — target pin number (0–15).

Parameters (inout)	None	—
Parameters (out)	state	uint8_t* — pointer where the pin logic level will be stored (0 = low, 1 = high).
Return value	GPIO_Status_t	GPIO_OK on success. GPIO_ERR_PORT, GPIO_ERR_PIN, or GPIO_ERR_NULL if any parameter is invalid.
Description	Reads the current logic level of a GPIO pin from the IDR register by shifting right by pin and masking with 0x1. Stores the result (0 or 1) at the address pointed to by state. Works on both input and output configured pins since IDR always reflects the actual pin voltage.	
Constraints	The state pointer must be non-NULL. The port clock must be enabled.	
Available via	GPIO_stm32f4.h	

4.4. Internal Function Definitions

The LCD driver defines four internal static helpers: `pcf8574_write()`, `lcd_pulse_enable()`, `lcd_send_nibble()`, `lcd_send_byte()`. Not exposed in `lcd_i2c.h`. Internal state is maintained exclusively through the two static constant arrays `gpio_ports[]` and `gpio_clk_masks[]` described in Section 3.3. No internal functions are subject to this section.

5. Future Enhancements

The following enhancements may be considered for future revisions of this LCD driver:

- Custom character support: `lcd_create_char()` to write 8-byte bitmaps to HD44780 CGRAM slots 0-7.FR[0] and AFR[1] register fields. This is required to enable peripheral functions such as UART, SPI, and I2C on GPIO pins.
- Newline handling: detect `\n` in `lcd_print()` and advance to the next row automatically.
- 10-bit I2C address support: extend `lcd_handle_t` to support PCF8574A variants at 0x38-0x3F.in.
- Fast Mode I2C: test at 400 kHz to reduce per-character write time from ~1.6ms to ~0.4ms.gh speed).
- DMA-based I2C: replace blocking `pcf8574_write()` with DMA transfers to free the CPU between writes.(rising, falling, both) and enabling the NVIC for the corresponding EXTI lines.
- `lcd_printf()`: add formatted print using an internal buffer and `utils_itoa()` for numeric output.R, which is more efficient for bus-style interfaces.
- ing accidental reconfiguration after initialization.
- lePin.
- 2F4 variants.